



# HANDS4EVENTS

## Manual Técnico

**Alumno:** Elías Pedrosa Madrid

**Tutor:** Pedro Vargas Pérez

CFGS DAM – IES Pablo Picasso | Curso 2025/2026

# 1. INTRODUCCIÓN

---

**Hands4Events** es una plataforma multiplataforma orientada a centralizar y optimizar la gestión del personal en el sector de montaje, logística y hostelería de eventos bajo contratación temporal. La plataforma está concebida para digitalizar procesos clave, ofreciendo un canal de comunicación y gestión directo, fiable y en tiempo real.

El sistema se divide en dos componentes principales diseñados bajo una misma base de código escalable:

- **Aplicación móvil (Android):** Dirigida al trabajador. Facilita la gestión de su día a día laboral, incluyendo revisión de próximos eventos, fichaje geolocalizado, gestión de disponibilidad, chat interno y descarga de nóminas.
- **Panel web de administración:** Dirigido a los coordinadores y empresas. Permite realizar el CRUD completo de trabajadores, creación y asignación de eventos, revisión de fichajes y gestión documental.

Este proyecto ha sido desarrollado como el **Producto Mínimo Viable (MVP)** para el Trabajo de Fin de Grado (TFG) del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM). Su arquitectura modular ha sido planteada desde su concepción para garantizar un mantenimiento ágil y una futura escalabilidad del sistema.

## 2. ANÁLISIS DEL PROBLEMA

---

### 2.1 Problemática

Tradicionalmente, los trabajadores del sector de eventos (montaje de escenarios, hostelería temporal, logística) carecen de herramientas digitales unificadas para el desempeño de su labor. Las agencias y empresas suelen depender de canales informales como grupos de WhatsApp, llamadas telefónicas o registros en papel para comunicar horarios, gestionar altas y bajas, o realizar el control de presencia.

Esta dependencia genera desinformación, pérdida de documentos (contratos, nóminas), dificultad para auditar las horas reales trabajadas y problemas de privacidad. Existe, por tanto, una necesidad inminente de centralizar, automatizar y digitalizar toda esta gestión operativa en una única plataforma.

### 2.2 Usuarios objetivo

La plataforma está orientada a resolver los problemas de dos perfiles clave:

- **Trabajadores eventuales:** Personal temporal que necesita conocer con claridad sus asignaciones, poder indicar su disponibilidad horaria, registrar su jornada de manera fiable y tener acceso rápido a su información salarial y laboral.
- **Empresas de personal y coordinadores de eventos:** Administradores que requieren gestionar equipos dinámicos y rotativos, asignar roles específicos, controlar el gasto por evento y distribuir documentación de forma segura.

## 3. DISEÑO DE LA SOLUCIÓN

### 3.1 Tecnologías utilizadas

El *stack* tecnológico ha sido seleccionado primando la escalabilidad, el rendimiento y la facilidad de mantenimiento multiplataforma:

- **Lenguajes de programación:**
  - Dart: Lenguaje principal para la lógica y UI en Flutter.
  - JavaScript / Node.js: Utilizado exclusivamente para el desarrollo de las automatizaciones en Cloud Functions.
  - Kotlin: Utilizado a nivel de código nativo (MainActivity) para la creación y gestión del canal de notificaciones en Android.
- **Framework Frontend:** Flutter
- **Entornos de Desarrollo (IDEs):** Android Studio y Visual Studio Code.
- **Backend y Cloud (Ecosistema Firebase):**
  - *Authentication*: Gestión de acceso e identidad de usuarios.
  - *Cloud Firestore*: Base de datos NoSQL para persistencia en tiempo real.
  - *Cloud Storage*: Almacenamiento seguro de archivos pesados (PDFs de nóminas, imágenes de perfil).
  - *Cloud Messaging (FCM)*: Servicio de envío de notificaciones push.
- **Gestor de estado:** Provider.

### Dependencias principales (pubspec.yaml)

```
firebase_core      # Núcleo de integración con Firebase
firebase_auth      # Autenticación de usuarios
cloud_firestore    # Interfaz de conexión con la base de datos
firebase_storage   # Subida y bajada de documentos
firebase_messaging # Recepción de tokens y notificaciones push
provider           # Inyección de dependencias y gestión de estado
geolocator         # Acceso a sensores GPS del dispositivo
table_calendar     # Renderizado del calendario interactivo
intl               # Internacionalización y formateo de fechas
url_launcher       # Apertura de enlaces externos o URLs de PDF
cached_network_image # Carga optimizada de avatares desde la red
flutter_local_notifications # Integración de notificaciones locales en SO
```

## 3.2 Arquitectura del sistema

El sistema está estructurado mediante un modelo por capas para desacoplar las responsabilidades, garantizando un código limpio (Clean Architecture).

### Arquitectura Externa

Describe el flujo de datos entre las plataformas cliente y la infraestructura en la nube:

- Los clientes (App Android y Panel Web) se conectan directamente a Firebase (Auth, Firestore, Storage, FCM) a través del SDK de Flutter.
- Las operaciones complejas o que requieren privilegios administrativos recaen sobre el servidor mediante **Cloud Functions**, las cuales actúan como *triggers* automáticos que reaccionan a los cambios que suceden dentro de Firestore o se ejecutan de forma programada.

### Arquitectura Interna (Capas en Flutter)

El código fuente de la aplicación se organiza bajo una estructura de directorios orientada a dominios y características:

- **screens/**: Interfaz de Usuario (UI). Capa de presentación visual que no contiene lógica de negocio pesada, limitándose a consumir el estado.
- **providers/**: Patrón de gestión de estado. Archivos que extienden ChangeNotifier y exponen la lógica reactiva y las mutaciones de datos hacia la UI.
- **services/**: Abstracción de la comunicación con Firebase u otras APIs externas. Centraliza las llamadas asíncronas de lectura y escritura.
- **models/**: Entidades de dominio. Clases puras de Dart que incluyen métodos para su serialización y deserialización contra Firestore (fromMap, toMap).
- **widgets/**: Componentes visuales reutilizables a lo largo de toda la aplicación (botones personalizados, tarjetas, modales).
- **core/**: Elementos transversales como la configuración del tema visual, paleta de colores, constantes de la aplicación y diccionarios de traducciones.

#### Flujo de datos estándar implementado:

Pantalla (UI) → Invoca método en Provider → Provider llama a Service → Service muta/lee en Firestore → Provider actualiza estado → Pantalla se reconstruye.

## 3.3 Modelos de datos

Las entidades de dominio principales mapeadas en la aplicación son:

- **User:** Representa tanto a trabajadores como a administradores. Campos: id, nombre, email, teléfono, dirección, rol, avatarUrl, idioma, notifActivadas, fcmToken, debeReiniciarPassword, fechaContratacion.
- **Evento:** Detalla un evento asignado. Campos: id, titulo, fechaInicio, fechaFin, ubicacion, descripcion, cobroPorHora, trabajadoresIds (lista), trabajadoresRoles (mapa).
- **Fichaje:** Registro de jornada asociado a un trabajador y evento. Campos: id, trabajadorId, eventId, entrada, salida, estado (enCurso/pausado/finalizado), ubicacionEntrada, ubicacionSalida, pausas (lista de objetos anidados con inicio/fin).
- **Nomina:** Representación del documento salarial. Campos: id, trabajadorId, mes, anio, mesNumero, sueldoBruto, sueldoNeto, horasTrabajadas, pdfUrl, estado, fechaGeneracion.
- **Notificacion:** Alerta generada para un usuario. Campos: id, trabajadorId, tipo, titulo, mensaje, timestamp, leida, datos.
- **Disponibilidad:** Bloques de tiempo indicados por el trabajador. Campos: id, trabajadorId, fecha, horaInicio, horaFin, aplicarRecurrente, diaSemana.
- **ChatMensaje:** Estructura de la mensajería interna. Campos: id, autorId, autorNombre, texto, timestamp.

## 3.4 Persistencia de datos – Estructura Firestore

La base de datos sigue una estructura NoSQL orientada a documentos, organizando la información en las siguientes colecciones raíz:

- **users:** Colección que almacena los documentos con la información del perfil y configuración de cada usuario.
- **eventos:** Almacena la configuración de cada evento.
  - *Subcolección interna:* **mensajes**. Anidada dentro de cada documento de evento específico para aislar el chat de cada evento y optimizar lecturas.
- **fichajes:** Registro histórico de todos los *clock-ins* y *clock-outs*, vinculando en el documento la ID del evento y la ID del trabajador.
- **notificaciones:** Bandeja de entrada del sistema de alertas push, filtrada en cliente por trabajadorId.
- **disponibilidad:** Documentos generados desde el calendario interactivo que los administradores pueden consultar para realizar las asignaciones.
- **nominas:** Registros contables que contienen principalmente la referencia directa (URL) al archivo almacenado físicamente en Firebase Storage.

## 3.5 Cloud Functions

Para garantizar la seguridad, automatizar procesos y aliviar la carga computacional de la aplicación cliente, se ha desarrollado un backend *serverless* en Node.js que expone las siguientes 5 funciones críticas:

- **onWorkerDeleted:** Función de limpieza (trigger onDelete sobre la colección users). Si un administrador elimina un trabajador, esta función borra automáticamente su identidad de Firebase Auth (revocando su acceso real) y recorre la colección eventos para desasignarlo de todas sus futuras responsabilidades operativas.
- **onNotificacionCreada:** Trigger onCreate sobre notificaciones. Al insertarse un documento nuevo en la base de datos, el servidor verifica si el campo notifActivadas del usuario destino es verdadero y, si es así, extrae su fcmToken para enviarle una notificación Push real al dispositivo del trabajador.
- **onMensajeCreado:** Trigger onCreate en la subcolección de mensajes de un chat. Detecta cuando un usuario escribe en un evento y automáticamente inserta documentos en la colección raíz de notificaciones para el resto de compañeros y administradores asignados a ese evento.
- **recordatoriosEventos:** Función programada tipo *Cron Job* (Pub/Sub). Se ejecuta todos los días a las 9:00 AM (hora local de Madrid). Realiza una consulta sobre los eventos que comenzarán en las próximas 24 horas y emite una alerta a los trabajadores asignados para evitar absentismos.
- **limpiarMensajesAntiguos:** Función programada de mantenimiento de base de datos. Identifica y elimina permanentemente los documentos de chat de aquellos eventos que hayan finalizado hace más de 45 días, optimizando así los costes de almacenamiento y lectura de Firestore.



## 4. CONSIDERACIONES TÉCNICAS

A lo largo del diseño y la implementación de Hands4Events, me enfrenté a diversos retos arquitectónicos y técnicos que requirieron soluciones específicas para poder garantizar la estabilidad y la correcta experiencia de usuario del MVP. A continuación, detallo las decisiones más relevantes:

### Sistema de Doble Rol en un solo código

Decidí aunar la plataforma de trabajadores y el panel administrativo en un único proyecto Flutter. El desafío aquí era enrutar correctamente al usuario inmediatamente después del login. Para ello, implementé un control que lee el campo rol (worker o admin) del documento de usuario obtenido de Firestore y verifica la constante `kIsWeb` del framework. Si el rol es admin y se accede desde un navegador web, el sistema redirige al AdminShell. Si el rol es worker, el enrutador lo envía al MainScaffold móvil, aislando por completo la UI de ambas experiencias.

### Expulsión en tiempo real (Logout forzado)

La seguridad es prioritaria; si una empresa despidе o elimina a un trabajador, este debe perder el acceso al instante. Resolví esto implementando un *listener* activo (`StreamSubscription`) sobre el propio documento del trabajador en Firestore desde el Provider de Autenticación. Si el documento desaparece (porque el administrador lo borró), el listener salta inmediatamente y ejecuta el método de *logout* en el dispositivo cliente, devolviendo al usuario a la pantalla de inicio de sesión, incluso si la app estaba abierta.

### Envío de notificaciones Push con la App cerrada

Emitir notificaciones seguras directamente desde una app cliente hacia el dispositivo de otro usuario es una mala práctica de seguridad y arquitectura. Mi solución fue delegar esta responsabilidad a la Cloud Function `onNotificacionCreada`. La aplicación cliente se limita a escribir un registro en la base de datos de Firestore. Es el servidor, en un entorno de máxima confianza, el que detecta esta inserción, evalúa los tokens FCM y emite la señal hacia los servicios de mensajería de Android.

## El flujo del "Primer Login Obligatorio"

Para gestionar las credenciales, el administrador crea el usuario generando una contraseña temporal. Me pareció crítico obligar al trabajador a asegurar su cuenta en su primera interacción. Añadí un flag booleano `debeReiniciarPassword` en el modelo `User`. Tras la validación en Firebase Auth, la app comprueba este flag; si es positivo, bloquea el acceso al *Dashboard* y renderiza un *scaffold* sin botón de retroceso que fuerza la actualización de la contraseña a través de la API de Firebase antes de continuar.

## Fichaje por Geolocalización (GPS)

El registro de *clock-in / clock-out* exigía integrar validación de ubicación. El reto fue gestionar asíncronamente los permisos de ubicación en tiempo de ejecución en Android y prevenir bloqueos si el usuario denegaba el acceso. Utilizando la librería `geolocator`, establecí un flujo en el que la app solicita el permiso; si el usuario lo deniega o el GPS está apagado físicamente, la aplicación advierte pero permite realizar el fichaje almacenando coordenadas nulas. Esto asegura que la operativa del trabajador no se paralice por problemas de hardware, dejando el registro marcado para revisión por el coordinador.